



From RNN → Transformers → NLP → LLMs → GenAI

A Complete Beginner's Roadmap to Generative AI Mastery

Who this is for: Someone who has completed ANN, CNN, RNN — and now wants to understand how ChatGPT, DeepSeek, and Qwen actually work, from the ground up. No shortcuts. Real theory with real examples.

Table of Contents

1. [Why RNNs Weren't Enough](#)
2. [NLP Basics — How Computers Read Text](#)
3. [Attention — The Core Idea](#)
4. [The Transformer Architecture — Full Breakdown](#)
5. [TensorFlow/Keras Code — Building a Transformer](#)
6. [From Transformers to LLMs](#)
7. [LLM Architectures Deep Dive — GPT, DeepSeek, Qwen](#)
8. [The Road to GenAI Mastery](#)

1. Why RNNs Weren't Enough

You have built RNNs. You know they process sequences one token at a time, carrying a hidden state forward. They work — but they have three fundamental problems that limited them at scale.

Problem 1: The Bottleneck

An RNN compresses the entire input sequence into a single fixed-size hidden vector before generating output.

"The student from Cameroon who studied hard and passed all exams ..."

$[h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots \rightarrow h_{50}]$

one vector

"... graduated"

By the time the network reaches the end of a long sentence, the beginning is compressed beyond recognition. The hidden state is a lossy summary of everything before it.

Analogy: Imagine you read a 50-page document, someone covers it, then asks you to answer questions from memory. The last few pages are clear. The beginning is hazy. That is the RNN bottleneck.

Problem 2: No Parallelism

RNNs are sequential — step 5 depends on step 4, which depends on step 3. A 512-token sequence requires 512 sequential operations. GPUs are designed for parallel computation — thousands of cores simultaneously. RNNs waste nearly all of them. Training is slow.

Problem 3: Vanishing Gradients

Even with LSTMs, gradients weaken travelling backward through hundreds of time steps. Very long-range dependencies — a pronoun referring to a noun 200 tokens earlier — are still difficult to learn reliably.

The Solution

In 2017 Google published "**Attention Is All You Need**" (Vaswani et al.) — arguably the most important paper in modern AI. The key insight:

Instead of processing tokens sequentially, let every token directly attend to every other token simultaneously.

This eliminates the bottleneck, enables full parallelism, and handles arbitrary-length dependencies — all at once.

2. NLP Basics — How Computers Read Text

Computers cannot read words. They need numbers. This section explains how text becomes numbers that a neural network can process.

2.1 Tokenisation

Tokenisation splits text into units called **tokens**. A token is not always a word.

```
Text: "I love deep learning!"
```

```
Word-level:      ["I", "love", "deep", "learning", "!"]
```

```
Subword (BPE):   ["I", "_love", "_deep", "_learn", "ing", "!"]
```

```
Character-level: ["I", " ", "l", "o", "v", "e", " ", "..."]
```

Modern LLMs use **subword tokenisation** — Byte Pair Encoding (BPE) or SentencePiece.

Why subword?

- Word-level: huge vocabulary, fails on new words ("ChatGPT" → unknown)
- Character-level: sequences too long, loses word structure
- Subword: balance — common words are single tokens, rare words split into known parts

```
"unbelievable" → ["un", "believ", "able"]
```

```
"Cameroon" → ["Camer", "oon"]
```

```
"Hello" → ["Hello"] (common → single token)
```

```
"12345" → ["123", "45"]
```

2.2 Token IDs

Each token gets an integer ID — its index in the vocabulary:

```
Vocabulary:
"I"         → 1
"love"      → 2
"deep"      → 3
"learning"  → 4
"!"         → 5

"I love deep learning!" → [1, 2, 3, 4, 5]
```

2.3 Token Embeddings

A raw integer like 4 carries no mathematical meaning — the number 4 is not "similar" to 3 even if "learning" is related to "deep". We fix this with **embeddings**.

An embedding layer is a lookup table. Each token ID maps to a dense vector of floats (e.g., 512 numbers). These vectors are learned during training.

```
token_id = 4 ("learning")
embedding → [0.23, -0.51, 0.87, 0.12, ..., 0.44] ← 512 numbers

token_id = 3 ("deep")
embedding → [0.21, -0.49, 0.85, 0.10, ..., 0.41] ← similar vector!

token_id = 1 ("I")
embedding → [0.95, 0.02, -0.33, 0.78, ..., -0.21] ← very different
```

After training, semantically similar words have similar vectors. This is the geometric structure of meaning:

```
king - man + woman ≈ queen
Paris - France + Germany ≈ Berlin
malaria - fever + rash ≈ measles (in a medical-trained model)
```

2.4 Positional Encoding

Embeddings tell the model **what** each token is — but not **where** it is.

```
"Dog bites man" vs "Man bites dog"
Same tokens. Different positions. Completely different meanings.
```

RNNs get position for free — they process tokens in order. Transformers process all tokens simultaneously, so position must be explicitly injected.

The original Transformer uses sinusoidal positional encodings — mathematical waves of different frequencies that produce a unique fingerprint for each position:

```
Position 0: PE = [sin(0/100000), cos(0/100000), sin(0/100002), ...]
Position 1: PE = [sin(1/100000), cos(1/100000), sin(1/100002), ...]
Position 2: PE = [sin(2/100000), cos(2/100000), sin(2/100002), ...]
```

Key property: the dot product between two position encodings decreases smoothly as the distance between positions increases — giving the model a natural sense of "how far apart" two tokens are.

Modern LLMs use **Rotary Position Embedding (RoPE)** — encodes relative position directly into the attention computation, generalises better to sequences longer than seen in training.

The final input to the Transformer:

```
Input_vector = Token_Embedding + Positional_Encoding
```

3. Attention — The Core Idea

Attention is the heart of the Transformer. Let's build understanding from scratch with a real example.

3.1 The Human Intuition

Read this sentence:

```
"The animal didn't cross the street because it was too tired."
```

When your brain processes the word "it", you automatically scan back to decide: does "it" refer to "animal" or "street"? You find "animal" is most likely because animals get tired, streets don't. You mentally assign high importance to "animal" and low importance to "street" when interpreting "it".

Attention does exactly this — mathematically. For each token, the model computes a score against every other token, uses those scores to create importance weights, and combines information from all other tokens proportionally.

3.2 Query, Key, Value (Q, K, V)

This is the notation that confuses nearly every beginner. Here is a concrete analogy first.

Library search system:

- **Query (Q):** What you are searching for. "I want books about African agriculture."
- **Key (K):** The label/metadata on each book. "Farming in Uganda", "French cuisine", "Ancient Egypt"
- **Value (V):** The actual content of each book.

The process:

1. Compare your Query against every Key → similarity scores
2. Softmax the scores → attention weights (sum to 1, all positive)
3. Retrieve: multiply each Value by its weight and sum → your answer

In the Transformer, Q, K, V all come from the same input — just projected differently:

```
Q = X @ W_Q    (X = input embeddings, W_Q = learned weight matrix)
K = X @ W_K
V = X @ W_V
```

3.3 Scaled Dot-Product Attention — Step by Step

```
Attention(Q, K, V) = softmax( Q × KT / √dk ) × V
```

Let's trace through a tiny example with 4 tokens: ["The", "cat", "sat", "mat"]

Each embedded as a 4-dimensional vector (simplified for illustration).

Step 1: Compute $Q \times K^T$ — the raw score matrix

```
Shape of Q: [4 tokens, 4 dims]
Shape of K: [4 tokens, 4 dims]
Shape of KT: [4 dims, 4 tokens]
```

```
Q × KT → shape [4, 4]
```

```
Each entry [i][j] = dot product of token i's query with token j's key
                    = "how much should token i attend to token j?"
```

	The	cat	sat	mat
The →	[5.1,	1.2,	0.3,	0.8]
cat →	[0.7,	8.4,	2.1,	0.4]
sat →	[0.2,	3.1,	7.5,	0.9]
mat →	[0.6,	0.5,	1.2,	6.8]

Step 2: Scale by $\sqrt{d_k}$

```
Divide every score by √4 = 2
```

```
Why? Large dk → large dot products → softmax becomes very "spiky"
(one near-1, rest near-0) → gradients vanish.
Dividing stabilises the gradient flow.
```

Step 3: Softmax across each row

```
Each row → probabilities that sum to 1
```

```
cat row after softmax (approximate):
```

```
["The": 0.04, "cat": 0.82, "sat": 0.11, "mat": 0.03]
```

```
Meaning: when building "cat"'s new representation,
82% of information comes from "cat" itself,
11% from "sat" (what the cat is doing),
4% from "The", 3% from "mat".
```

Step 4: Multiply by V

Output for "cat" = $0.04 \times V["The"] + 0.82 \times V["cat"] + 0.11 \times V["sat"] + 0.03 \times V["mat"]$

The output is a weighted sum of all value vectors.

High attention weight → that token contributes more to the output.

The full result: Every token now has a new vector that incorporates information from every other relevant token — based on learned similarity. "it" in a sentence will attend heavily to the noun it refers to. "bank" will attend to context words that disambiguate river vs financial.

3.4 Multi-Head Attention

A single attention computation forces the model to mix all relationships into one representation. Multi-Head Attention runs **H attention operations in parallel**, each learning different relationship types:

Head 1 might learn: syntactic relationships (subject → verb)
Head 2 might learn: coreference (pronoun → noun it refers to)
Head 3 might learn: positional (nearby tokens)
Head 4 might learn: semantic similarity (synonyms)
Head 5-8 might learn: relationships the model discovers on its own

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_H) \times W_O$

where $\text{head}_i = \text{Attention}(Q \times W_{Q_i}, K \times W_{K_i}, V \times W_{V_i})$

Each head has its own learned projection matrices in smaller dimension:

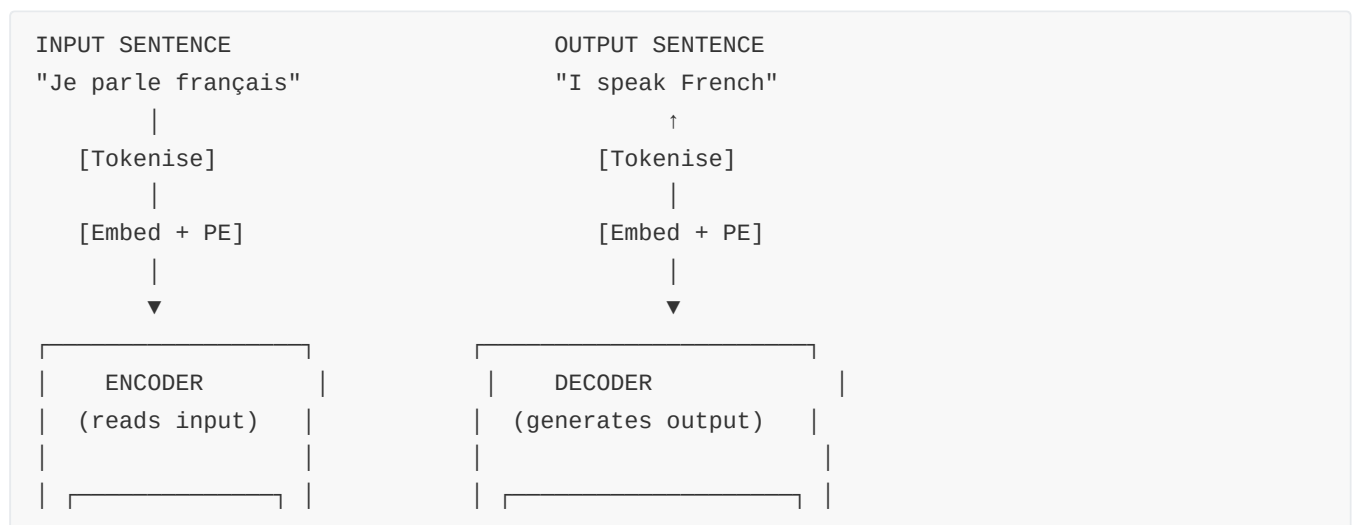
$d_{\text{head}} = d_{\text{model}} / \text{num_heads}$

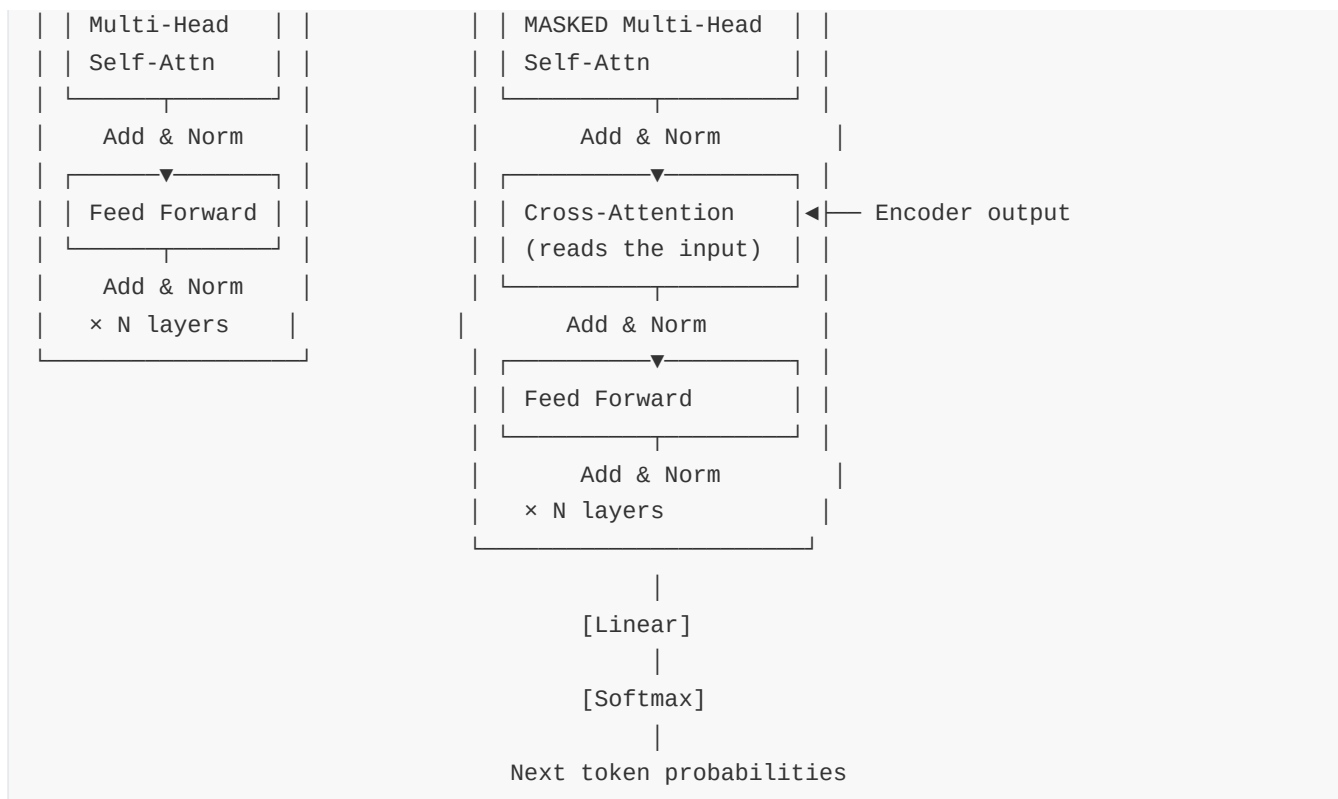
If $d_{\text{model}} = 512$ and $\text{num_heads} = 8$, each head works in 64 dimensions. Total computation is the same as a single head — just split into 8 specialised sub-problems.

4. The Transformer Architecture — Full Breakdown

Now we put it all together. The original Transformer (for translation) has an Encoder and a Decoder.

4.1 Visual Overview





4.2 Encoder Block (repeated N times)

Sub-layer 1 — Multi-Head Self-Attention:

Every token attends to every other token in the input. "bank" in "river bank" vs "bank account" will have different contextual representations after this step.

Add & Norm — Residual connection + Layer Normalisation:

```
output = LayerNorm(x + SubLayer(x))
```

The residual `x + SubLayer(x)` is critical. It gives gradients a clean path backward through every layer, preventing vanishing gradients in deep networks. Think of it as a shortcut highway alongside the main road — even if the main road has traffic, the highway stays clear.

Sub-layer 2 — Feed-Forward Network (FFN):

Two linear layers with GELU activation, applied identically to each token position independently:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1) W_2 + b_2$$

`d_ff` is typically $4 \times d_{\text{model}}$:

`d_model = 512` → `d_ff = 2048`

The FFN is where most of the model's **factual knowledge is stored**. Think of attention as the routing system (deciding who talks to whom) and the FFN as the knowledge vault (storing and retrieving facts).

4.3 Decoder Block (Encoder + Extra)

The decoder has all encoder components plus:

Masked Self-Attention: When generating token at position 5, the decoder cannot look at positions 6, 7, 8... (they haven't been generated yet). The causal mask enforces this:

Mask for 4 tokens – 1 = can attend, 0 = blocked:

T1

T2

T3

T4

T1

[

1

0

0

0

]

← sees only itself

T2

[

1

1

0

0

]

← sees T1 and itself

T3

[

1

1

1

0

]

T4

[

1

1

1

1

]

← sees everything before it

0 positions → set to $-\infty$ before softmax → become 0 after softmax

Cross-Attention: The decoder's Queries come from the decoder hidden states. The Keys and Values come from the encoder output. This is how the decoder "reads" the encoded source sentence when generating each target word.

4.4 Layer Normalisation

LayerNorm(x) = $\gamma \times (x - \mu) / (\sigma + \epsilon) + \beta$

μ = mean of x across d_model (feature dimension)

σ = std of x across d_model

γ, β = learned scale and shift

Unlike BatchNorm (normalises across the batch), LayerNorm normalises across features for a single sample — works stably for variable-length sequences and small batches.

Modern LLMs use **RMSNorm** (Root Mean Square Norm) — removes the mean-centering step, 10-15% faster with equivalent performance:

RMSNorm(x) = $x / \sqrt{\text{mean}(x^2) + \epsilon} \times \gamma$

4.5 Key Hyperparameters — Scale Comparison

Parameter	Our small model	GPT-2 Small	GPT-3	GPT-4 (est.)
d_model	128	768	12,288	~25,000
num_heads	4	12	96	~160
num_layers	2	12	96	~120
d_ff	512	3,072	49,152	~100,000
vocab_size	20,000	50,257	50,257	~100,000
Total params	~2M	117M	175B	~1.8T
Training tokens	—	40B	570B	15T+

5. TensorFlow / Keras Code

5.1 Positional Encoding

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

def positional_encoding(max_seq_len, d_model):
    """
    Sinusoidal positional encodings.
    Returns shape [1, max_seq_len, d_model]
    """
    positions = np.arange(max_seq_len)[:, np.newaxis]    # [seq_len, 1]
    dims      = np.arange(d_model)[np.newaxis, :]        # [1, d_model]

    angles = positions / np.power(10000, (2*(dims//2)) / d_model)
    angles[:, 0::2] = np.sin(angles[:, 0::2])    # even dims → sin
    angles[:, 1::2] = np.cos(angles[:, 1::2])    # odd  dims → cos

    return tf.cast(angles[np.newaxis, ...], tf.float32) # [1, seq_len, d_model]

# Visualise
PE = positional_encoding(50, 64)
plt.figure(figsize=(12, 4))
plt.pcolormesh(PE[0].numpy(), cmap='RdBu')
plt.colorbar()
plt.title('Positional Encoding – each row = one position')
plt.xlabel('Embedding Dimension')
plt.ylabel('Position in Sequence')
plt.tight_layout()
plt.savefig('positional_encoding.png', dpi=150)
plt.show()
```

5.2 Scaled Dot-Product Attention

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Q: [batch, heads, seq_q, d_k]
    K: [batch, heads, seq_k, d_k]
    V: [batch, heads, seq_v, d_v]
    mask: [batch, 1, 1, seq_k] – 1=attend, 0=block
    """
    d_k = tf.cast(tf.shape(K)[-1], tf.float32)
    scores = tf.matmul(Q, K, transpose_b=True) / tf.math.sqrt(d_k)
    # scores: [batch, heads, seq_q, seq_k]

    if mask is not None:
        scores += (1.0 - mask) * (-1e9)    # masked positions → -∞ → 0 after softmax

    weights = tf.nn.softmax(scores, axis=-1)
```

```

output = tf.matmul(weights, V)
return output, weights

```

5.3 Multi-Head Attention Layer

```

from tensorflow import keras

class MultiHeadAttention(keras.layers.Layer):
    def __init__(self, d_model, num_heads, **kwargs):
        super().__init__(**kwargs)
        assert d_model % num_heads == 0
        self.num_heads = num_heads
        self.d_model = d_model
        self.d_k = d_model // num_heads

        self.W_Q = keras.layers.Dense(d_model, use_bias=False)
        self.W_K = keras.layers.Dense(d_model, use_bias=False)
        self.W_V = keras.layers.Dense(d_model, use_bias=False)
        self.W_O = keras.layers.Dense(d_model, use_bias=False)

    def split_heads(self, x):
        """[batch, seq, d_model] → [batch, heads, seq, d_k]"""
        batch = tf.shape(x)[0]
        seq = tf.shape(x)[1]
        x = tf.reshape(x, [batch, seq, self.num_heads, self.d_k])
        return tf.transpose(x, [0, 2, 1, 3])

    def call(self, x, context=None, mask=None):
        if context is None:
            context = x # self-attention

        Q = self.split_heads(self.W_Q(x))
        K = self.split_heads(self.W_K(context))
        V = self.split_heads(self.W_V(context))

        attn_out, weights = scaled_dot_product_attention(Q, K, V, mask)

        # Merge heads: [batch, heads, seq, d_k] → [batch, seq, d_model]
        batch = tf.shape(attn_out)[0]
        seq = tf.shape(attn_out)[2]
        attn_out = tf.transpose(attn_out, [0, 2, 1, 3])
        attn_out = tf.reshape(attn_out, [batch, seq, self.d_model])

        return self.W_O(attn_out), weights

```

5.4 Feed-Forward Network + Encoder Layer

```

class FeedForward(keras.layers.Layer):
    def __init__(self, d_model, d_ff, dropout_rate=0.1, **kwargs):
        super().__init__(**kwargs)
        self.dense1 = keras.layers.Dense(d_ff, activation='gelu')

```

```

self.dense2 = keras.layers.Dense(d_model)
self.dropout = keras.layers.Dropout(dropout_rate)

def call(self, x, training=False):
    return self.dense2(self.dropout(self.dense1(x), training=training))

class EncoderLayer(keras.layers.Layer):
    def __init__(self, d_model, num_heads, d_ff, dropout_rate=0.1, **kwargs):
        super().__init__(**kwargs)
        self.attn = MultiHeadAttention(d_model, num_heads)
        self.ffn = FeedForward(d_model, d_ff, dropout_rate)
        self.norm1 = keras.layers.LayerNormalization(epsilon=1e-6)
        self.norm2 = keras.layers.LayerNormalization(epsilon=1e-6)
        self.drop1 = keras.layers.Dropout(dropout_rate)
        self.drop2 = keras.layers.Dropout(dropout_rate)

    def call(self, x, mask=None, training=False):
        # Sub-layer 1: Self-attention + residual
        attn_out, _ = self.attn(x, mask=mask)
        x = self.norm1(x + self.drop1(attn_out, training=training))

        # Sub-layer 2: FFN + residual
        x = self.norm2(x + self.drop2(self.ffn(x, training=training), training=training))
        return x

```

5.5 Full Encoder + Sentiment Classifier (IMDB)

```

class TransformerEncoder(keras.layers.Layer):
    def __init__(self, num_layers, d_model, num_heads, d_ff,
                 vocab_size, max_seq_len, dropout_rate=0.1, **kwargs):
        super().__init__(**kwargs)
        self.d_model = d_model
        self.embedding = keras.layers.Embedding(vocab_size, d_model)
        self.pos_enc = positional_encoding(max_seq_len, d_model)
        self.enc_layers = [EncoderLayer(d_model, num_heads, d_ff, dropout_rate)
                           for _ in range(num_layers)]
        self.dropout = keras.layers.Dropout(dropout_rate)

    def call(self, x, mask=None, training=False):
        seq_len = tf.shape(x)[1]
        x = self.embedding(x) * tf.math.sqrt(tf.cast(self.d_model, tf.float32))
        x += self.pos_enc[:, :seq_len, :]
        x = self.dropout(x, training=training)
        for layer in self.enc_layers:
            x = layer(x, mask=mask, training=training)
        return x

# — Build sentiment classifier —————
VOCAB_SIZE = 20000; MAX_LEN = 256
D_MODEL = 128; NUM_HEADS = 4; D_FF = 512; NUM_LAYERS = 2

(X_train, y_train), (X_test, y_test) = keras.datasets.imdb.load_data(num_words=VOCAB_SIZE)

```

```

X_train = keras.preprocessing.sequence.pad_sequences(X_train, maxlen=MAX_LEN,
padding='post')
X_test  = keras.preprocessing.sequence.pad_sequences(X_test,  maxlen=MAX_LEN,
padding='post')
X_val=X_train[-5000:]; y_val=y_train[-5000:]
X_train=X_train[:-5000]; y_train=y_train[:-5000]

inputs  = keras.Input(shape=(MAX_LEN,), dtype=tf.int32)
encoder = TransformerEncoder(NUM_LAYERS, D_MODEL, NUM_HEADS, D_FF,
                             VOCAB_SIZE, MAX_LEN, dropout_rate=0.1)
x        = encoder(inputs, training=False)
x        = keras.layers.GlobalAveragePooling1D()(x)  # pool over sequence
x        = keras.layers.Dense(64, activation='relu')(x)
x        = keras.layers.Dropout(0.3)(x)
outputs = keras.layers.Dense(1, activation='sigmoid')(x)

model = keras.Model(inputs, outputs, name='TransformerSentiment')
model.summary()
print(f"Parameters: {model.count_params():,}")

model.compile(optimizer=keras.optimizers.Adam(1e-4),
              loss='binary_crossentropy',
              metrics=['accuracy', keras.metrics.AUC(name='auc')])

history = model.fit(
    X_train, y_train, epochs=20, batch_size=64,
    validation_data=(X_val, y_val),
    callbacks=[keras.callbacks.EarlyStopping(
        monitor='val_auc', patience=5, restore_best_weights=True, mode='max')],
    verbose=1
)

_, test_acc, test_auc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_acc:.4f} | Test AUC: {test_auc:.4f}")
# Expected: ~88-91% – better than simple LSTM

```

5.6 Causal Decoder (GPT-style Text Generation)

```

def create_causal_mask(seq_len):
    """Lower triangular mask – token i can only see tokens 0..i."""
    mask = tf.linalg.band_part(tf.ones([seq_len, seq_len]), -1, 0)
    return mask[tf.newaxis, tf.newaxis, :, :] # [1,1,seq_len,seq_len]

class DecoderOnlyTransformer(keras.Model):
    def __init__(self, num_layers, d_model, num_heads, d_ff,
                 vocab_size, max_seq_len, dropout_rate=0.1):
        super().__init__()
        self.embedding = keras.layers.Embedding(vocab_size, d_model)
        self.pos_enc    = positional_encoding(max_seq_len, d_model)
        self.dec_layers = [EncoderLayer(d_model, num_heads, d_ff, dropout_rate)
                           for _ in range(num_layers)]
        self.dropout    = keras.layers.Dropout(dropout_rate)

```

```

self.out_proj = keras.layers.Dense(vocab_size)
self.d_model = d_model

def call(self, x, training=False):
    seq_len = tf.shape(x)[1]
    causal_mask = create_causal_mask(seq_len)
    x = self.embedding(x) * tf.math.sqrt(tf.cast(self.d_model, tf.float32))
    x += self.pos_enc[:, :seq_len, :]
    x = self.dropout(x, training=training)
    for layer in self.dec_layers:
        x = layer(x, mask=causal_mask, training=training)
    return self.out_proj(x) # [batch, seq, vocab] logits

# Training: predict next token at every position
# Input: [tok0, tok1, tok2, tok3]
# Target: [tok1, tok2, tok3, tok4] (shifted by 1)

def generate(model, token_ids, max_new=100, temperature=1.0):
    """Autoregressive generation – one token at a time."""
    for _ in range(max_new):
        ids = tf.constant([token_ids[-512:]]) # context window
        logits = model(ids, training=False) # [1, seq, vocab]
        next_logit = logits[0, -1, :] / temperature # last position
        next_id = tf.random.categorical(
            next_logit[tf.newaxis, num_samples=1][0, 0].numpy()
        )
        token_ids.append(int(next_id))
    return token_ids

```

6. From Transformers to LLMs

You have seen a small Transformer. An LLM is:

Transformer + enormous scale + enormous data + careful training recipe

6.1 The Scale That Changes Everything

Dimension	GPT-2 (2019)	GPT-3 (2020)	GPT-4 (2023)	DeepSeek-V3 (2024)
Parameters	1.5B	175B	~1.8T	671B (37B active)
Training tokens	40B	570B	15T+	14.8T
Training cost	\$40k	\$4.6M	\$100M+	~\$5.5M

6.2 The Training Recipe

Phase 1 — Pre-training (self-supervised):

The model trains on trillions of tokens of text — books, web pages, code, papers, Wikipedia. The task is simple: predict the next token. No human labels needed.

Input: "The capital of Cameroon is"
Target: "Yaoundé"

Through billions of such predictions, the model learns:

- Grammar and syntax
 - World facts
 - Reasoning patterns
 - Programming logic
 - Mathematical relationships
- All from next-token prediction alone.

Phase 2 — Supervised Fine-Tuning (SFT):

Human experts write (instruction, ideal response) pairs. The model is fine-tuned to follow instructions rather than just complete text.

Before SFT: "Write a poem about malaria"
Model output: "... write a poem about malaria symptoms the fever..."
(continues the sentence, doesn't write a poem)

After SFT: A proper poem is generated.

Phase 3 — RLHF (Reinforcement Learning from Human Feedback):

- Step 1: Model generates two responses (A and B) to the same prompt
Step 2: Human rater says "A is better than B"
Step 3: A reward model is trained on thousands of such preferences
Step 4: The LLM is trained with RL to maximise the reward model's score
Step 5: Repeat 100,000+ times

Result: Model becomes more helpful, more honest, less harmful

6.3 Emergent Abilities

A remarkable phenomenon: abilities that are absent in small models appear suddenly at scale — without being explicitly programmed.

Ability	Appears at approximately
Few-shot learning (learn from examples)	~7B parameters
Chain-of-thought reasoning	~100B
Multi-step arithmetic	~100B
Code generation	~10B
Language translation (untrained)	~100B

No one programmed chain-of-thought reasoning into GPT-3. It emerged from scale.

6.4 The Context Window Problem

The context window is how many tokens the model can "see" at once:

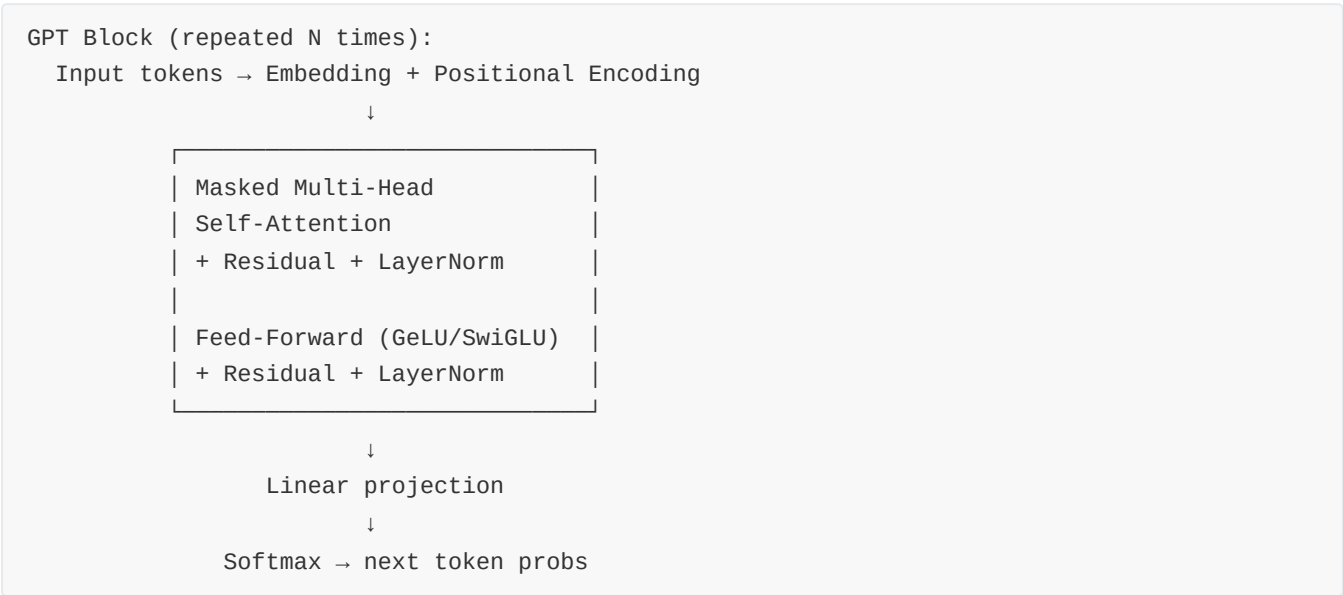
Model	Context Window	Equivalent
GPT-2	1,024 tokens	~¾ page
GPT-3	4,096 tokens	~3 pages
GPT-4 Turbo	128,000 tokens	~200 pages
Gemini 1.5 Pro	1,000,000 tokens	~1,500 pages

The challenge: standard attention is $O(n^2)$ in sequence length. Doubling the context makes attention 4× more expensive. Modern techniques like **Flash Attention**, **Sliding Window Attention**, and **Ring Attention** address this.

7. LLM Architectures Deep Dive

7.1 GPT — OpenAI (Decoder-only)

GPT's insight: for text generation, you only need the decoder. Remove the encoder entirely.



GPT evolution:

Version	Year	Parameters	Key Innovation
GPT-1	2018	117M	Pre-train → fine-tune paradigm
GPT-2	2019	1.5B	Quality shock — "too dangerous to release"
GPT-3	2020	175B	Few-shot learning without fine-tuning
InstructGPT	2022	175B	RLHF — follows instructions reliably

Version	Year	Parameters	Key Innovation
GPT-4	2023	~1.8T	Multimodal, 128k context, likely MoE
GPT-4o	2024	unknown	Native audio + video + text unified

GPT-4 is believed to be a Mixture of Experts (MoE):

Instead of one 1.8T model, GPT-4 is estimated to be ~8 expert models of ~220B each. A routing network activates only 2 experts per token:

Total params: 1.8T (all experts combined)
Active params: ~440B (only 2 of 8 experts per token)

Benefit: massive model capacity with manageable inference cost

7.2 DeepSeek — DeepSeek AI

DeepSeek matched GPT-4 class quality at a fraction of the training cost. Their key architectural innovations:

Multi-Head Latent Attention (MLA):

Standard attention stores full Key and Value matrices for every token in the KV cache — this grows linearly with context length and consumes enormous GPU memory.

Standard MHA KV cache per token:
= num_heads × d_k × 2 (K and V)
= 128 × 128 × 2 = 32,768 numbers per token
× 128,000 context = 4 billion numbers – huge

MLA: compress K and V into a small latent vector first
KV cache per token = d_compressed (e.g., 512 numbers)
At inference: latent → K, V via learned projection
Memory reduction: ~5-13×

Mixture of Experts at scale:

Property	DeepSeek-V3
Total parameters	671B
Active per token	37B (only 2 experts active at once)
Number of experts	256 fine-grained experts
Context window	128K tokens
Training tokens	14.8 trillion
Training cost	~\$5.5M (vs \$100M+ for GPT-4)

Why so cheap to train?

1. **FP8 mixed-precision** — 8-bit floats halve memory bandwidth vs 16-bit
2. **Auxiliary-loss-free load balancing** — MoE models usually need extra loss terms to spread load across experts. DeepSeek solved this without extra loss
3. **Multi-token prediction** — simultaneously predict next 2-4 tokens → better data efficiency
4. **DualPipe parallelism** — new distributed training algorithm reducing GPU idle time

DeepSeek-R1 — Reasoning through RL:

Standard path to reasoning (RLHF):
Pre-train → SFT (imitate human reasoning examples) → RL (refine)

DeepSeek-R1 path:
Pre-train → RL with verifiable rewards only (math, code)
→ model discovers chain-of-thought reasoning by itself
→ no human demonstrations needed

Significance: reasoning behaviour emerged from reward signal alone

7.3 Qwen — Alibaba Cloud

The Qwen series (0.5B to 72B+) covers text, images, video, audio, code, and math.

Key architectural components in Qwen2.5:

Grouped Query Attention (GQA):

Multi-Head Attention (MHA): each head has unique Q, K, V
→ maximum quality, maximum memory
→ 8 heads → 8 KV pairs in cache

Multi-Query Attention (MQA): all heads share 1 K and V
→ minimum memory, quality drops
→ 8 heads → 1 KV pair in cache

Grouped Query Attention (GQA): groups of heads share K and V
→ balanced quality and memory
→ 8 heads, 2 groups → 2 KV pairs in cache

Qwen uses GQA: 4× less KV cache memory vs MHA, ~same quality

SwiGLU Activation:

Used in Qwen, LLaMA, Mistral, DeepSeek — most modern LLMs. Replaces the standard ReLU FFN:

Standard FFN: $\text{output} = \text{ReLU}(xW_1) \times W_2$
 $\wedge$ simple threshold

SwiGLU FFN: $\text{output} = (\text{Swish}(xW_1) \odot xW_3) \times W_2$
 where $\text{Swish}(x) = x \times \text{sigmoid}(x)$
 and $\odot$ = element-wise multiplication

xW_3 acts as a learned gate controlling how much of xW_1 passes through.
This gating mechanism improves perplexity by ~1-2% with no extra inference cost.

Rotary Position Embedding (RoPE):

Used in Qwen, LLaMA, Mistral, DeepSeek, Falcon — essentially all modern open models.

Instead of adding a positional vector to the embedding, RoPE rotates the Q and K vectors:

Standard attention: $\text{score} = Q \cdot K$
RoPE attention: $\text{score} = (R_m Q) \cdot (R_n K)$

where R_m = rotation matrix for position m

Key insight: $(R_m Q) \cdot (R_n K)$ depends ONLY on $(m-n)$ – the relative distance.
Not on m and n separately (absolute positions).

Why this matters: the model naturally learns relative distances.
Generalises better to longer sequences than seen during training.

Qwen2.5-72B at a glance:

Property	Value
Architecture	Decoder-only Transformer
Parameters	72 billion
Attention	Grouped Query Attention
Position encoding	RoPE
Activation	SwiGLU
Context window	128K tokens
Vocabulary	151,646 tokens
Training tokens	18 trillion
Performance	Matches or beats GPT-4 on many benchmarks
License	Apache 2.0 (open weights)

7.4 Architecture Comparison Table

Feature	GPT-4	DeepSeek-V3	Qwen2.5-72B	LLaMA 3.1-70B
Type	Decoder MoE	Decoder MoE	Decoder dense	Decoder dense
Total params	~1.8T	671B	72B	70B
Active params/token	~440B	37B	72B	70B
Attention type	Sparse MHA	MLA	GQA	GQA
Position encoding	Learned	RoPE	RoPE	RoPE
FFN activation	GeLU	SwiGLU	SwiGLU	SwiGLU
Normalisation	Pre-RMSNorm	Pre-RMSNorm	Pre-RMSNorm	Pre-RMSNorm
Context window	128K	128K	128K	128K
Open weights	✗	✓	✓	✓
Training cost	~\$100M+	~\$5.5M	Not disclosed	~\$10M

8. The Road to GenAI Mastery

Your Complete Learning Roadmap

STAGE 1 – Foundation ✓ (complete)

Python, NumPy, Pandas, Matplotlib

Linear algebra, calculus, probability

Machine learning fundamentals

ANN, CNN, RNN with TensorFlow/Keras

STAGE 2 – Transformers & NLP ✓ (this document)

Tokenisation and embeddings

Attention mechanism from scratch

Full Transformer architecture

Building in TF/Keras

Understanding LLMs

STAGE 3 – Working with Pre-trained LLMs

Hugging Face Transformers library

Running open models (Qwen, LLaMA, Mistral)

Prompt engineering

Structured output / function calling

Evaluating LLM quality

STAGE 4 – Fine-tuning

Full fine-tuning (when you have GPU budget)

LoRA / QLoRA – efficient fine-tuning

Dataset preparation for instruction tuning

RLHF / DPO – preference alignment

STAGE 5 – RAG (Retrieval Augmented Generation)

- Vector databases (Chroma, Pinecone, Weaviate)
- Embedding models
- Chunking strategies
- Reranking
- Evaluation of RAG pipelines

STAGE 6 – Agents and Tools

- ReAct framework
- Function/tool calling
- LangChain / LlamaIndex
- Multi-agent systems
- Memory and state management

STAGE 7 – Production

- Serving (vLLM, TGI, FastAPI)
- Quantisation (GGUF, AWQ, GPTQ)
- Monitoring and observability
- Cost optimisation
- Responsible AI deployment

Stage 3 — Quick Start Code

```
# pip install transformers accelerate bitsandbytes

from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM
import torch

# — Option A: Pipeline – fastest way to use any LLM —
generator = pipeline('text-generation', model='Qwen/Qwen2.5-7B-Instruct',
                    device_map='auto')
result = generator(
    [{"role": "user", "content": "Explain malaria in simple terms."}],
    max_new_tokens=200
)
print(result[0]['generated_text'])

# — Option B: Full control —
tokenizer = AutoTokenizer.from_pretrained('Qwen/Qwen2.5-7B-Instruct')
model = AutoModelForCausalLM.from_pretrained(
    'Qwen/Qwen2.5-7B-Instruct',
    torch_dtype=torch.float16,
    device_map='auto'
)

messages = [
    {"role": "system", "content": "You are a helpful medical assistant."},
    {"role": "user", "content": "What are the symptoms of malaria?"}
]
```

```

text      = tokenizer.apply_chat_template(messages, tokenize=False)
inputs    = tokenizer(text, return_tensors='pt').to(model.device)
outputs   = model.generate(**inputs, max_new_tokens=300, temperature=0.7,
                           do_sample=True)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

Stage 4 — LoRA Fine-tuning

```

# pip install peft transformers datasets trl

from peft import get_peft_model, LoraConfig, TaskType
from transformers import AutoModelForCausalLM

# Load base model
model = AutoModelForCausalLM.from_pretrained('Qwen/Qwen2.5-7B')

# Configure LoRA
# Instead of updating W (7B×d weights), add two small matrices A and B
# New weights = W + α × (A × B)
# W is frozen, only A and B are trained
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                # rank – controls quality vs efficiency
    lora_alpha=32,        # scaling factor
    lora_dropout=0.1,
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj']
)

peft_model = get_peft_model(model, lora_config)
peft_model.print_trainable_parameters()
# → trainable params: 83M || all params: 7.6B || trainable%: 1.09%
# Only 1% of parameters trained – massive memory saving

```

LoRA explained visually:

Standard fine-tuning:

$$W_{\text{new}} = W_{\text{original}} + \Delta W \quad (\Delta W \text{ has same size as } W - \text{huge})$$

LoRA:

$$W_{\text{new}} = W_{\text{original}} + \alpha \times (A \times B)$$

W: [8192, 8192] – frozen, 67M params
A: [8192, r=16] – trained, 131K params
B: [r=16, 8192] – trained, 131K params
New params: 262K (vs 67M) – 99.6% reduction in trainable params

Stage 5 — RAG Pipeline

```

# pip install langchain chromadb sentence-transformers

from langchain.embeddings import HuggingFaceEmbeddings

```

```

from langchain.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains import RetrievalQA

# Your documents – e.g., WHO malaria guidelines
documents = [
    "WHO recommends artemisinin-based combination therapies (ACTs) as first-line treatment...",
    "Children under 5 in high-transmission areas should receive seasonal malaria chemoprevention...",
    "Diagnosis should be confirmed by microscopy or rapid diagnostic test before treatment...",
]

# Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.create_documents(documents)

# Embed and store
embeddings = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')
vectordb = Chroma.from_documents(chunks, embeddings)

# Query
retriever = vectordb.as_retriever(search_kwargs={'k': 3})
# When user asks a question:
# 1. Embed the question
# 2. Find 3 most similar chunks in the database
# 3. Inject them into the prompt as context
# 4. LLM answers based on retrieved context – not hallucinated memory

```

Project Ideas by Stage

Stage 3 – Working with LLMs:

- ✦ Medical FAQ chatbot using Qwen + WHO documents
- ✦ French/English language assistant for students
- ✦ Code explainer for Python beginners

Stage 4 – Fine-tuning:

- ✦ Fine-tune LLaMA on Nigerian medical consultation transcripts
- ✦ Instruction-tune Qwen to answer questions in Fulfulde
- ✦ Code assistant fine-tuned on African tech community Q&A

Stage 5 – RAG:

- ✦ Agricultural advisor RAG with farming guides from IITA/CGIAR
- ✦ Legal aid bot with RAG over Cameroonian law documents
- ✦ School curriculum assistant with RAG over textbooks

Stage 6 – Agents:

- ✦ Agent that reads weather + crop prices + gives farmer advice
- ✦ Medical triage agent: symptom collection → diagnosis → referral
- ✦ Research assistant that searches, reads papers, summarises

Stage 7 – Production:

- ✦ Deploy a fine-tuned model on a Raspberry Pi for offline village use
- ✦ SMS-based LLM access for feature phones (no smartphone needed)
- ✦ Low-bandwidth optimised API for rural internet connectivity

Mental Model Summary

SEQUENCE MODELLING EVOLUTION:

RNN (1986):	Sequential, forgets early context
LSTM (1997):	Gating → longer memory, still sequential
Attention (2015):	Direct token-to-token connections (added to RNN)
Transformer (2017):	Attention-only, fully parallel, any-distance connections
BERT (2018):	Encoder-only, bidirectional → understanding tasks
GPT (2018):	Decoder-only, left-to-right → generation tasks
GPT-3 (2020):	175B → emergent few-shot reasoning
ChatGPT (2022):	GPT-3.5 + RLHF → mainstream adoption
LLaMA (2023):	Open weights → democratised research
DeepSeek-V3 (2024):	MLA + MoE → GPT-4 quality at 3% training cost
Qwen2.5 (2024):	GQA + SwiGLU + 18T tokens → best open 72B
DeepSeek-R1 (2025):	Pure RL → chain-of-thought emerges without demonstrations

KEY BUILDING BLOCKS (memorise these):

- Token Embedding + Positional Encoding (RoPE in modern models)
- Multi-Head Attention (or GQA, MLA)
- Feed-Forward Network (SwiGLU)
- Residual Connections + RMSNorm
- Causal Mask (for decoder / generation)
- Mixture of Experts (efficiency at trillion-parameter scale)

THE TRAINING RECIPE:

1. Pre-train on trillions of tokens → next-token prediction
2. SFT on instruction pairs → follow instructions
3. RLHF or DPO → align with human values
4. Optional RL for reasoning (DeepSeek-R1 approach)

YOUR PATH FORWARD:

- ✓ ANN → CNN → RNN (foundation)
- ✓ Transformers + LLM theory (this document)
 - Hugging Face – run and prompt LLMs
 - LoRA fine-tuning on domain data
 - RAG for knowledge-grounded applications
 - Agents that act in the world
 - Production deployment and optimisation
 - Research frontier: new architectures, alignment, multimodality

Further Reading

Paper	Why read it
Vaswani et al. (2017) — <i>Attention Is All You Need</i>	The original Transformer — arxiv.org/abs/1706.03762
Brown et al. (2020) — <i>Language Models are Few-Shot Learners</i>	GPT-3 — few-shot emergence
Hu et al. (2021) — <i>LoRA: Low-Rank Adaptation of LLMs</i>	Efficient fine-tuning
Su et al. (2021) — <i>RoFormer: Rotary Position Embedding</i>	RoPE
Ainslie et al. (2023) — <i>GQA: Generalised Multi-Query Transformers</i>	GQA
DeepSeek-AI (2024) — <i>DeepSeek-V3 Technical Report</i>	MLA + MoE
Qwen Team (2024) — <i>Qwen2.5 Technical Report</i>	Modern open LLM
DeepSeek-AI (2025) — <i>DeepSeek-R1: Incentivising Reasoning via RL</i>	RL for reasoning